# Adaptive KV-Cache Placement for Tiered-Memory LLM Inference: Halving Tail Latency at HBM-to-DRAM Crossings

Mahima Sachan, Sarah Pradhan, Sahil Vanjara
Department of Computer Science and Engineering
California State University, Long Beach
CECS 530: Advanced Computer Architecture
May 9, 2026
Mahima.Sachan01@student.csulb.edu, Sarah.Pradhan01@student.csulb.edu, Sahil.Vanjara01@student.csulb.edu,

***Abstract*—Autoregressive Large Language Model (LLM) decoding often operates near the memory-bandwidth roofline rather than the peak-compute roofline. As contexts grow, the key/value (KV) cache can outgrow on-package GPU memory and require placement in a lower-bandwidth tier such as host DRAM or CXL-attached memory. A capacity-reactive placement policy concentrates the migration cost at the boundary step, creating a tail-latency spike even though the total bytes moved are unchanged. This paper introduces a novel *adaptive KV-placement policy* that treats migration *timing*—not migration volume—as the primary optimization variable: the same bytes are moved as under a capacity-reactive policy, but spread across $K$ preceding decode steps rather than concentrated at the boundary crossing. The policy is implemented as a post-hoc analyzer over a four-tier memory simulator (SRAM/HBM/DRAM/FAR_MEM) tightly coupled to a modified `llama-cpp-python` inference pipeline; the simulator reads KV occupancy directly from the engine's `n_tokens` counter after each `eval` call, grounding the analytical traffic model in actual runtime state rather than synthetic assumptions. On Llama-2-7B at the HBM→DRAM boundary ($\sim$32 GB KV-cache), adaptive placement with $K = 16$ reduces the worst-case decode-step latency from $1011.8$ ms to $538.1$ ms, a 46.8% reduction, with no measured change in total decode time. We also validate the memory-bound trend on two NVIDIA accelerators with a $4.86\times$ HBM-bandwidth ratio (A100-SXM4-40GB at 1.555 TB/s and T4 at 0.32 TB/s) and three Q4-quantized models. Llama-2-7B's measured throughput ratio is $4.21\times$ (87% of the bandwidth-ratio prediction), while smaller models show weaker scaling because fixed runtime overhead is a larger fraction of each decode step. Finally, we compare KV quantization, sliding-window attention, and FlashAttention-style fusion under a common bytes-per-token model. The complete artifact is packaged as a single Colab notebook.**

***Index Terms*—Tiered memory, LLM inference, KV-cache, adaptive placement, HBM, CXL, tail latency, roofline, memory bandwidth**

## I. Introduction

Autoregressive LLM decoding is commonly limited by memory movement rather than by the accelerator's advertised floating-point throughput. Each decoded token streams model weights and reads the KV-cache state accumulated so far; in a roofline view, this gives an arithmetic intensity (AI) near $2 \cdot N_{\text{params}}/b_{\text{weights}}$ for the weight-dominated portion of decode. On the Q4-quantized models in our experiments, the measured AI falls in the 2–3 FLOP/byte range, far below the ridge points of the tested GPUs and consistent with prior memory-centric analyses of LLM inference [1], [2], [8].

The KV-cache adds a second, context-length-dependent pressure point. Its size grows linearly with the number of tokens, layers, and KV heads. For Llama-2-7B [14], a context near 65,000 tokens produces a KV-cache of roughly 32 GB in the simulator, which is close to the capacity threshold we reserve for HBM after accounting for weights and runtime buffers. Past this threshold, cache blocks must be modeled in a lower-bandwidth tier such as host DRAM or a CXL-attached pool. Our static baseline abstracts representative capacity-reactive behavior in engines such as vLLM and `llama.cpp`: cache placement changes only after the current tier can no longer hold the cache, so migration work is charged to the crossing step [3], [6].

For human-facing generation workloads such as chat, coding assistants, and tool-using agents, inter-token latency and tail latency are part of the user-visible service budget. A boundary step that pays hundreds of milliseconds of migration cost can dominate the perceived latency profile even if the average decode cost remains unchanged. We therefore study a capacity-aware policy that predicts the tier crossing from KV growth and starts migration before the boundary, spreading the same bytes over earlier decode steps instead of concentrating them in one stall.

**This paper makes the following contributions:**

- **An adaptive KV-cache placement policy** parameterised by a lookahead window $K$ that begins migration before HBM exhaustion. We compare it with a capacity-reactive static baseline inspired by the placement behavior exposed by current open-source runtimes [3], [6]. On Llama-2-7B at the HBM→DRAM crossing, adaptive placement at $K = 16$ reduces the worst-case decode-step latency from **1011.8 ms to 538.1 ms (46.8% reduction)** with no impact on total decode time. (Section IV, Sec. VI-B.)

- **A four-tier hierarchical memory simulator** (SRAM, HBM, DRAM, FAR_MEM) integrated as a modified inference pipeline atop `llama-cpp-python`, providing capacity-aware placement, migration tracking, bandwidth-bound latency, and per-byte energy modeling. The pipeline drives both the static policy (baseline) and our adaptive policy. (Section III, Section V.)
- **Cross-bandwidth empirical validation** across two NVIDIA accelerators spanning a $4.86\times$ memory-bandwidth ratio (A100-SXM4-40GB, T4) using vendor-published hardware specifications [16], [17]. Measured throughput ratio matches the bandwidth ratio within 13% on Llama-2-7B, supporting the memory-bound interpretation on real hardware. (Sec. VI-C.)
- **A unified bytes-per-token comparison** of KV quantization, sliding-window attention, and FlashAttention-style fusion – across all three models. Sliding-window $W = 512$ delivers the largest reduction (30%) for Llama-2-7B at our test context. (Sec. VI-E.)
- **A reproducible, single-notebook artifact.** The complete code (simulator, pipeline, experiments, figures) executes end-to-end on a Colab Pro A100 and is published as a public GitHub repository.

## II. Background and Related Work

### A. Memory-Bound LLM Inference

The Roofline model of Williams et al. [1] compares measured performance with a piecewise-linear ceiling determined by peak compute and peak memory bandwidth. The ridge point separates compute-bound and bandwidth-bound operating regions. Using NVIDIA's published A100-SXM4-40GB specifications (312 BF16/FP16 Tensor TFLOP/s and 1.555 TB/s HBM bandwidth), the ridge is approximately 200 FLOP/byte [16]. Our dense decoder-only measurements cluster around 2–3 FLOP/byte, well below that ridge.

Prior LLM-serving work reaches the same qualitative conclusion from different angles. FlexGen showed that weight-stationary scheduling is poorly matched to batch-size-one autoregressive generation [2]. Gholami et al. describe the widening gap between model-size growth and memory-bandwidth growth, which motivates treating bandwidth as a first-order inference bottleneck [8].

### B. KV-Cache Management

The KV-cache has become a first-class resource. PagedAttention [3] introduced virtual memory primitives to KV management, supporting batched serving without contiguous allocation. KVQuant [9] and StreamingLLM [10] reduce KV traffic via aggressive quantization and sliding-window attention sinks respectively. These works treat the KV-cache as residing in a single tier (HBM) with a single capacity bound; the question of *what happens when that bound is exceeded* is largely orthogonal and is what this paper addresses.

### C. Tiered and Disaggregated Memory Hardware

The simulator uses representative tier parameters rather than a single commercial node. For the HBM tier we use a modern accelerator-class bandwidth of 3.35 TB/s, consistent with H100-class HBM3 subsystems, while the measured A100 experiments use the A100's 1.555 TB/s bandwidth [16], [18]. The DRAM tier is modeled with DDR5-class bandwidth (68 GB/s), and the far-memory tier uses a CXL-attached pool with substantially lower effective bandwidth and higher access latency [11], [19]. Under these assumptions, the HBM-to-CXL bandwidth ratio is greater than $130\times$. Prior work such as TPP shows that CXL pools can help capacity-bound, latency-tolerant workloads [4]; our focus is the decode-step penalty in an interactive LLM setting.

### D. Tail-Latency-Aware Memory Tiering

Outside the LLM context, tail-latency-aware memory tiering has a substantial body of work. Lim et al. [15] proposed disaggregated memory blade pre-fetching to hide remote-access latency; TPP [4] used PMU-based access-frequency sampling to drive transparent page placement under CXL. Our adaptive policy is, in spirit, an LLM-specific instance of this proactive-tiering family: instead of relying on access frequency (uninformative for the autoregressive write-once-read-many KV pattern), we exploit the closed-form predictability of KV growth ($\Delta k = b_{\text{KV-write}}$ per token) to schedule migrations deterministically.

### E. The Modified-Pipeline Choice

We implement our proposed policy using a modified inference pipeline with explicit memory placement. This approach is preferred over a purely analytical simulation for three reasons: (i) wrapping a real runtime preserves end-to-end functional correctness, (ii) it enables validation of the analytical traffic model against real wall-clock latencies on physical hardware, and (iii) it produces reproducible results that can be verified via a single, integrated Colab notebook.

## III. System Architecture

### A. Four-Tier Memory Topology

We model a four-tier memory hierarchy with bandwidth, latency, capacity, and per-byte energy parameters drawn from public hardware references. Table I summarizes the configuration.

TABLE I
Four-Tier Memory Hierarchy

| Tier | BW | Latency | Cap. | Contents |
|---|---|---|---|---|
| T0 SRAM | 10 TB/s | 1 ns | 4 MB | Hot KV, activations, logits |
| T1 HBM | 3.35 TB/s | 10 ns | 32 GB | Model weights + warm KV |
| T2 DDR5 | 68 GB/s | 80 ns | 64 GB | Overflow KV (long context) |
| T3 CXL 3.0 | 25 GB/s | 500 ns | 512 GB | Disaggregated pool, cold KV |

### B. Static Placement Policy (Baseline)

Our baseline is a capacity-reactive policy consistent with the placement abstractions exposed by current open-source inference engines, but simplified so the migration cost is explicit:

- **Weights** are placed in the fastest tier with capacity $\geq w_{\text{bytes}}$, skipping Tier 0 (no real LLM weight tensor fits in 4 MB).
- **KV-cache** is placed in the fastest tier with capacity $\geq k_{\text{bytes}}(t)$ at the current decode step. As the cache grows, the baseline moves it to the next tier only when the capacity test fails; *the modeled migration is therefore charged at the crossing step*.
- **Compute buffers** (activations, attention intermediates, logits) live in Tier 0, consistent with FlashAttention-style tiling.

The static policy's defining property is not the total number of bytes moved, but the timing of those bytes. It charges the bulk migration, $\tau_{\text{mig}} = k_{\text{bytes}}/B(\text{TIER}_{\text{dest}})$, to one decode step. For Llama-2-7B at the HBM→DRAM crossing this is $32.4\,\text{GB}/68\,\text{GB/s} \approx 476$ ms, added to the normal latency of that step.

### C. Bytes per Decode Step

For a single decoded token at KV size $T$:

$$b_{\text{weight}} = w_{\text{bytes}} \tag{1}$$

$$b_{\text{KV-write}} = L \cdot 2 \cdot H_{kv} \cdot d_{\text{head}} \cdot e_{\text{KV}} \tag{2}$$

$$b_{\text{KV-read}} = T \cdot b_{\text{KV-write}} \tag{3}$$

$$b_{\text{tot}}(T) = b_{\text{weight}} + b_{\text{KV-read}} + b_{\text{KV-write}} \tag{4}$$

where $L$ is layer count, $H_{kv}$ is the KV-head count (GQA-aware), $d_{\text{head}} = d_{\text{model}}/H$, and $e_{\text{KV}} \in \{2.0, 1.0, 0.5\}$ bytes per KV element under fp16, int8, or int4 quantization. Weights dominate at short context; KV reads grow linearly in $T$ and overtake weights once $T \cdot 2H_{kv}d_{\text{head}}e_{\text{KV}} \gtrsim w_{\text{bytes}}/L$. The closed-form crossover is $T^* = w_{\text{bytes}}/(L \cdot 2H_{kv}d_{\text{head}}e_{\text{KV}})$.

### D. Latency and Energy Model

Per step we charge bandwidth-bound latency at the placed tier:

$$\tau_{\text{step}}(T) = \frac{b_{\text{weight}}}{B(\text{TIER}_w)} + \frac{b_{\text{KV-read}}}{B(\text{TIER}_{kv})} + \frac{b_{\text{KV-write}}}{B(\text{TIER}_{kv})} \tag{5}$$

with $B(\cdot)$ from Table I. Per-byte energy is charged at the placed tier; MAC energy uses Horowitz' 0.5 pJ/op figure for fp16 tensor cores [12]. The model intentionally ignores per-access latency and pipeline overlap, giving a clean lower bound that we validate against measured wall-clock decode in Sec. VI.

### E. Traffic-Reduction Optimizations (Modeled)

We model three traffic-reduction techniques:

**O1:** **KV quantization.** Reduce $e_{\text{KV}}$ from 2.0 (fp16) to 1.0 (int8) or 0.5 (int4). Reduces $b_{\text{KV}}$ proportionally and defers tier migration by the same factor.

**O2:** **Sliding-window attention.** Replace $b_{\text{KV-read}}(T) = T \cdot b_{\text{KV-write}}$ with $b^{\text{win}}_{\text{KV-read}}(T) = \min(T, W) \cdot b_{\text{KV-write}}$ for window $W$. Caps KV traffic at a constant.

**O3:** **Operator fusion (FlashAttention-style) [5].** Eliminates the $O(T^2)$ $\mathbf{QK}^\top$ and softmax round-trips to HBM during prefill. Decode (Q of length 1) is already $O(T)$, so fusion's saving is concentrated on the prefill phase.

## IV. Adaptive KV-Placement Policy

### A. Motivation

Under the static placement policy of Sec. III, moving a 32 GB KV-cache from HBM to DRAM at 68 GB/s adds roughly 476 ms to the boundary step. In the Llama-2-7B crossing experiment, that step reaches **1011.8 ms** (Sec. VI-B). Subsequent steps still pay the slower DRAM bandwidth for KV reads; adaptive placement does not change that steady-state cost. The policy target is narrower: remove the extra migration spike at the boundary by scheduling the transfer earlier.

### B. Mechanism

We propose an adaptive placement policy parameterized by a lookahead window $K$:

1) Track the per-step KV growth rate $\Delta k$. Under autoregressive decode with batch size 1, $\Delta k = b_{\text{KV-write}}$, which is constant and known a-priori from the model architecture.
2) At each step, compute the predicted overflow time: $n_{\text{overflow}} = \lceil (C(\text{TIER}) - k_{\text{cur}})/\Delta k \rceil$.
3) When $n_{\text{overflow}} \leq K$, schedule an incremental background migration of $k_{\text{cur}}/n_{\text{overflow}}$ bytes per step. The runtime executes these copies in parallel with the weight reads of the current decode step (which fully saturate the HBM read pipeline under static placement, leaving spare write bandwidth on the destination tier).
4) At the crossing step, the cache is already resident in the destination tier; no in-line bulk transfer is required.

The policy does not reduce the migration volume; it changes only when the same $k_{\text{cur}}$ bytes are copied. In the simulated traces, total decode time is unchanged within rounding (0.0% overhead in Sec. VI-B), while the maximum boundary-step latency decreases.

**Choice of $K$ = 16.** The lookahead window trades boundary-spike height against per-step pre-migration overhead. A small $K$ (e.g., $K = 4$) is insufficient: at the HBM→DRAM crossing each amortized step still incurs $476\,\text{ms}/4 = 119$ ms of extra overhead, leaving a spike nearly as tall as the static case. Conversely, a large $K$ (e.g., $K = 64$) begins copying sixty-four steps in advance; beyond $K \approx 16$ the residual worst-case step is bounded by the post-crossing DRAM-bandwidth decode cost (∼500 ms), so further spreading the migration yields diminishing returns while adding unnecessary bookkeeping. At $K = 16$ each pre-migration step adds ≈30 ms—a modest increment—while the worst-case boundary step drops from 1011.8 ms to 538.1 ms.

### C. Pseudo-Code

```
def analyze_static_vs_adaptive(sim, K=16):
  static_lat = [r.dec_ms for r in sim.step_log]
  adaptive_lat = [r.dec_ms for r in sim.step_log]
  prev = sim.step_log[0].kv_tier
  for r in sim.step_log:
    if r.kv_tier != prev: # crossing
      mig_ms = r.kv_size / B[r.kv_tier]
      # static: full stall at the crossing step
      static_lat[r.step-1] += mig_ms
      # adaptive: spread over preceding K steps
      for s in range(max(0, r.step-K), r.step):
        adaptive_lat[s] += mig_ms / K
      prev = r.kv_tier
  return static_lat, adaptive_lat
```

### D. Why Adaptive Placement Is the Right Lever

The placement decision has both a spatial component (which tier stores the cache) and a temporal component (when migration begins). The static baseline fixes the temporal component at the crossing step. The adaptive policy exposes it through $K$, making the boundary-latency profile tunable while preserving the same total migration work. Sec. VI-B reports how this affects the tail-latency profile at tier crossings.

## V. Implementation

### A. Software Stack

The artefact is a single Jupyter notebook executable on Google Colab. It imports `llama-cpp-python` [13] (the Python bindings for `llama.cpp` [6]) and uses the low-level `Llama` object's `tokenize`, `eval`, `sample`, and `n_tokens` APIs. The KV-cache size used by the simulator is read directly from `llm.n_tokens` after each `eval` call, so the analytical traffic count tracks the real engine state token-for-token.

The modified inference pipeline wraps the engine:

```
toks = llm.tokenize(prompt.encode(), add_bos=True)
t0 = time.perf_counter()
llm.eval(toks) # prefill
prefill_ms = (time.perf_counter() - t0)*1000
sim.record_prefill(len(toks), llm.n_tokens)

t_d = time.perf_counter()
for _ in range(max_new): # decode
  tok = llm.sample(temp=0.0)
  if tok == llm.token_eos(): break
  llm.eval([tok])
  sim.record_decode_step(llm.n_tokens)
decode_ms = (time.perf_counter() - t_d)*1000
```

### B. Hardware

We collected real measurements on two NVIDIA accelerators provisioned via Google Colab Pro. Table II summarizes the configuration using vendor-published peak specifications for memory bandwidth and tensor throughput [16], [17]. The 4.86× memory-bandwidth ratio is the central design lever for the cross-bandwidth validation in Sec. VI-C.

TABLE II
Cross-Bandwidth Hardware Configuration

| **Property** | **A100-SXM4-40GB** | **Tesla T4** |
|---|---|---|
| Peak tensor FP16/BF16 where available (TFLOPS) | 312 | 65 |
| HBM bandwidth (TB/s) | 1.555 | 0.32 |
| VRAM (GB) | 40 | 16 |
| Roofline ridge (FLOP/B) | 200 | 203 |

### C. Models

We use three open GGUF checkpoints quantized with Q4_K_M (Table III). Q4_K_M is a block-based `llama.cpp`/GGUF weight-quantization format; the effective on-disk model size includes quantized weights plus per-block scale and metadata overhead [6], [24].

TABLE III
Models Under Test (all Q4_K_M)

| **Model** | $L$ | $H$ | $H_{kv}$ | $d_{\text{model}}$ | $w_{bytes}$ |
|---|---|---|---|---|---|
| SmolLM2-135M | 30 | 9 | 3 | 576 | 104 MB |
| TinyLlama-1.1B | 22 | 32 | 4 | 2048 | 670 MB |
| Llama-2-7B | 32 | 32 | 32 | 4096 | 4.08 GB |

### D. Chat Templating

Each model is wrapped in its documented chat template before tokenization: ChatML-style markers for SmolLM2, the Zephyr-style format used by TinyLlama-1.1B-Chat, and the Llama-2 `[INST]` wrapper for Llama-2-Chat [20]–[23]. Without this wrapper, the instruction-tuned checkpoints can terminate immediately on raw prompts, producing empty decode traces. The template strings are taken from the published tokenizer configuration or model cards; the weights and tokenizers are otherwise unchanged.

### E. Reproducibility

All sampling is greedy ($T = 0$); seeds are pinned to $\{42, 1337, 2024\}$. Outputs are GPU-tagged so dual-GPU runs do not overwrite each other. Code, model-download URLs, and reproduction instructions are at https://github.com/mahima110298/Adaptive-KV-Cache-Placement-for-Tiered-Memory-LLM-Inference.git.

## VI. Experimental Setup and Results

### A. Experimental Matrix

**Real measurements:** 3 models × 3 contexts $\{1024, 2048, 4096\}$ × 2 prompt kinds × 3 seeds, run separately on A100 and T4 (108 measured cells nominally; see "Data quality note" below).
**Simulator-only sweep:** 3 models × 8 contexts (128 to 2,097,152 tokens) × 3 KV-quants (16, 8, 4 bits).
**Optimization comparison:** 5 variants × 3 models, all at ctx $= 4096$.
**Adaptive vs static:** 2 models (TinyLlama-1.1B, Llama-2-7B) at the smallest context that forces an HBM→DRAM crossing (∼65,792 tokens for 7B).

### B. Adaptive vs. Static KV-Placement (Headline)

Figure 1 shows the per-step latency profile around the HBM→DRAM crossing for Llama-2-7B and TinyLlama-1.1B under both policies. For Llama-2-7B, the static policy produces a single tall stall at step 65,409 – the bulk transfer of 32.4 GB at DRAM bandwidth, totaling 1011.8 ms in one step. The adaptive policy spreads the same work across $K = 16$ preceding decode steps, reducing the worst-case step latency to 538.1 ms.

Table IV reports the headline metrics. The 46.8% tail-latency reduction comes *at zero overhead on total decode time* – the same migration bytes are moved, just rescheduled. Note that the post-crossing steady-state latency (∼500 ms per step at the new tier's bandwidth) is unchanged; the policy only eliminates the *stall at the boundary*, not the structural cost of the slower tier itself.

TABLE IV
ADAPTIVE VS. STATIC KV-PLACEMENT (LLAMA-2-7B, $K = 16$)

| Metric | Static | Adaptive | Reduction |
|---|---|---|---|
| Tail latency (ms) | 1011.8 | 538.1 | **46.8%** |
| Total decode time (s) | 546.21 | 546.21 | 0.0% |
| Migration steps | 1 | 1 | — |
| KV at crossing (GB) | 32.4 | 32.4 | — |
| Lookahead $K$ (steps) | — | 16 | — |

For TinyLlama-1.1B at the smaller SRAM→HBM boundary (KV at crossing ≈ 4 MB), the migration cost is sub-millisecond and the two policies are indistinguishable. This is consistent with the analytical prediction: the adaptive policy only matters when the migration cost dominates a single decode step's normal latency, which happens at the larger HBM→DRAM and DRAM→FAR_MEM boundaries.

### C. Cross-Bandwidth Validation of the Memory-Bound Thesis

Figure 2 places every measured decode step on the host roofline. AI clusters at 2.5–3.3 FLOP/byte across all three models on both A100 and T4, two orders of magnitude below the respective ridge points (200 and 203 FLOP/byte).

Table V summarises the cross-bandwidth tracking. If decode were perfectly bandwidth-bound, measured throughput would scale with the memory-bandwidth ratio ($1.555/0.32 = 4.86\times$). Llama-2-7B reaches a $4.21\times$ A100/T4 throughput ratio, or 87% of that prediction. TinyLlama and SmolLM2 scale less strongly ($2.22\times$ and $1.60\times$ respectively). We interpret the gap as fixed per-step overhead—Python↔C++ calls, kernel launches, sampling, and scheduler work—becoming more visible when the bytes moved per step are small. The largest model is therefore the cleanest empirical case for the memory-bound interpretation.

TABLE V
CROSS-BANDWIDTH THROUGHPUT TRACKING

| Model | A100 tok/s | T4 tok/s | A100/T4 ratio | vs. BW ratio (4.86×) |
|---|---|---|---|---|
| Llama-2-7B | 154.9 | 36.8 | 4.21× | 87% |
| TinyLlama-1.1B | 424.3 | 191.0 | 2.22× | 46% |
| SmolLM2-135M | 454.7 | 283.9 | 1.60× | 33% |

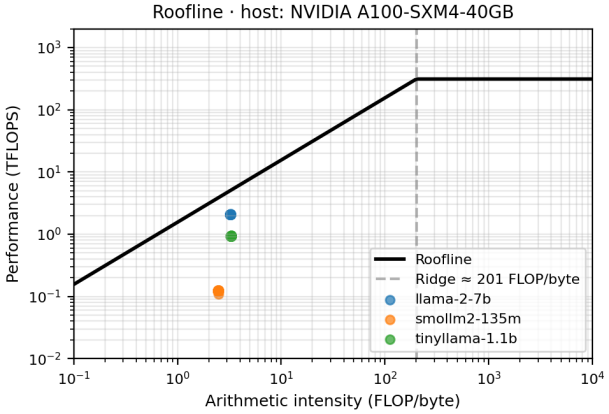


Fig. 2. Roofline against the host A100 GPU. Every measured decode step falls in the memory-bound region by approximately two orders of magnitude. The same clustering holds on T4 (not shown).

### D. Compute-Centric vs. Memory-Centric Prediction (Bonus)

Table VI compares two analytical predictions of decode throughput against measurement, on all three models on both GPUs. A compute-only model $\text{tps}_{\text{compute}} = P_{\text{peak}}/(2N_{\text{params}})$ overestimates measured Llama-2-7B throughput by $\mathbf{150}\times$ on A100 and $\mathbf{133}\times$ on T4 – and overestimates the smaller models by even larger factors (up to $2{,}544\times$ for SmolLM2). A reader who looks at peak TFLOPS to estimate decode throughput would be qualitatively wrong on every model, every GPU.

The memory-bound prediction $\text{tps}_{\text{memory}} = B_{\text{HBM}}/b_{\text{tot}}(T)$ is much closer for the largest model – a $\sim 2\times$ overestimate on Llama-2-7B, consistent with kernels achieving ∼45% of peak HBM bandwidth (the same fraction on both A100 and T4). For smaller models, the memory-bound prediction also overestimates more, again pointing to non-bandwidth overhead dominating the per-step budget at small bytes/step. Figure 3 visualises the gap on the bar chart corresponding to Table VI.

TABLE VI
COMPUTE-CENTRIC VS. MEMORY-CENTRIC PREDICTION ERROR

| Model | GPU | Cmp-only (t/s) | Mem-bound (t/s) | Meas. (t/s) | Cmp over-est. |
|---|---|---|---|---|---|
| Llama-2-7B | A100 | 23,284 | 371 | 154.9 | 150× |
| Llama-2-7B | T4 | 4,851 | 76 | 36.8 | 133× |
| TinyLlama-1.1B | A100 | 141,818 | 2,309 | 424.3 | 334× |
| TinyLlama-1.1B | T4 | 29,545 | 475 | 191.0 | 155× |
| SmolLM2-135M | A100 | 1,155,556 | 14,085 | 454.7 | 2,544× |
| SmolLM2-135M | T4 | 240,741 | 2,898 | 283.9 | 1,052× |

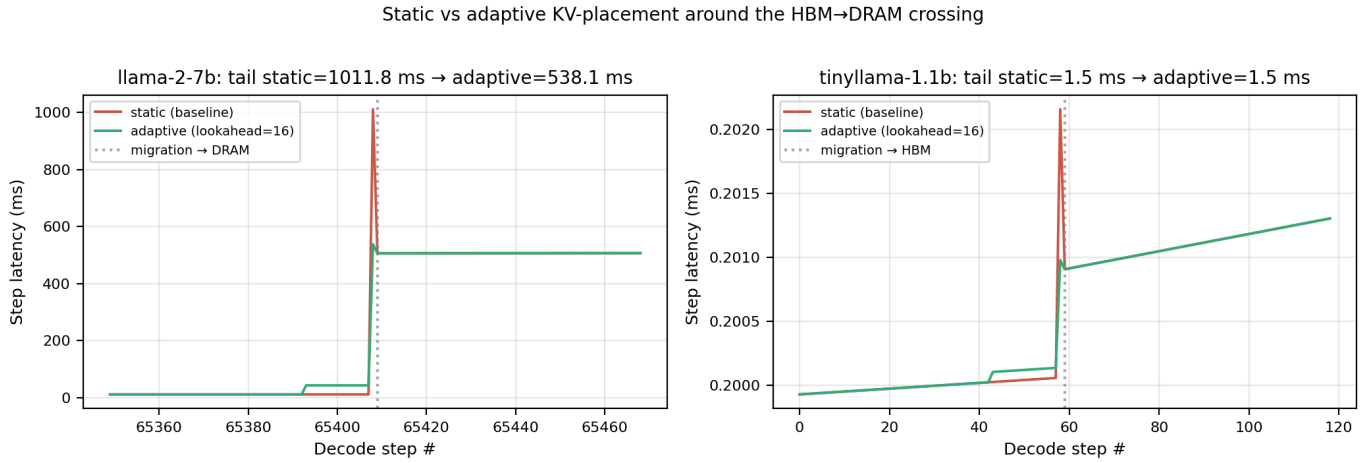


Fig. 1. Static vs. adaptive KV-placement at two tier boundaries. *Right:* Llama-2-7B at the HBM→DRAM crossing (32.4 GB migration). Static (red) pays the full migration cost in a single decode step (1011.8 ms); adaptive (green) spreads it across $K = 16$ preceding steps, dropping the worst-case step to 538.1 ms (46.8% reduction). *Left:* TinyLlama-1.1B at the SRAM→HBM crossing (4 MB migration). The two policies are indistinguishable, confirming the adaptive policy correctly adds zero overhead when the migration cost is already sub-millisecond – a negative-control case.

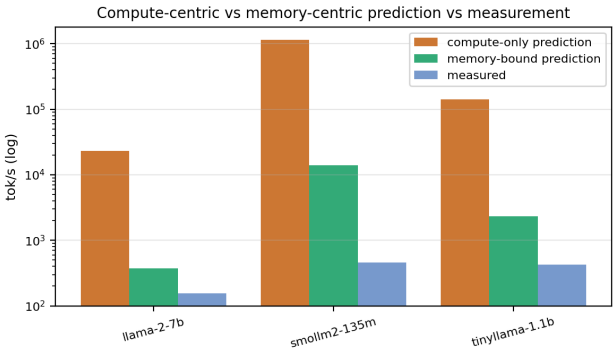


Fig. 3. Compute-only and memory-bound predictions vs. measured decode throughput. Compute-only over-predicts by two orders of magnitude on every model.

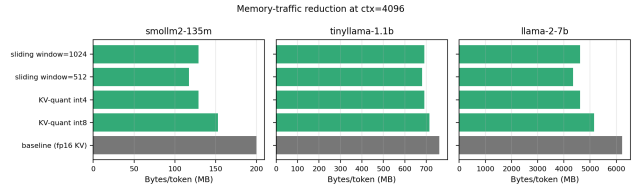


Fig. 4. Bytes per decoded token under each modeled optimization, ctx $=$ 4096. For Llama-2-7B, sliding-window $W = 512$ gives the largest reduction (30.2%); KV-int4 cuts 25.9%.

## E. Optimization Impact

Figure 4 compares baseline (fp16 KV) bytes/token against KV-int8, KV-int4, and sliding-window attention with $W \in \{512, 1024\}$, across all three models at ctx $= 4096$. For Llama-2-7B, sliding-window $W = 512$ delivers the largest reduction (30.2% versus baseline), with KV-int4 close behind (25.9%). KV-int8 is more conservative (17.2%) but preserves attention quality. Operator fusion (FlashAttention) eliminates a 2.1 GB $QK^\top$ round-trip per long-prompt prefill but does not affect the decode-phase bytes/token (Q has length 1).

## F. KV-Cache Growth and Tier Crossings

Figure 5 shows the simulator-only KV-growth curve for Llama-2-7B at three KV-quant settings. With fp16 KV, the cache crosses HBM (32 GB) at ∼65k tokens and DRAM (64 GB) at ∼130k. Quantizing KV to int4 defers each crossing by $4\times$. KV quantization is a placement-policy lever, not just a traffic reduction.

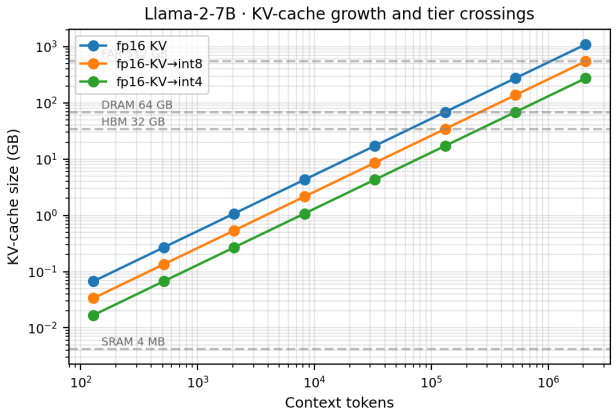


Fig. 5. Llama-2-7B KV-cache size vs. context length under three KV-quant settings. Horizontal dashed lines mark tier capacities. Quantizing fp16→int4 defers each tier crossing by 4× in context length.

### G. Bytes per Decoded Token (Real Measurements)

Figure 6 shows per-token memory traffic across the three models in our matrix. The curves are dominated by the constant weight term until KV reads catch up; for Llama-2-7B at ctx = 4096, KV reads are still under 7 % of the total per-token traffic, so the workload sits squarely in the weight-streaming regime at the contexts we can measure on Colab GPUs.

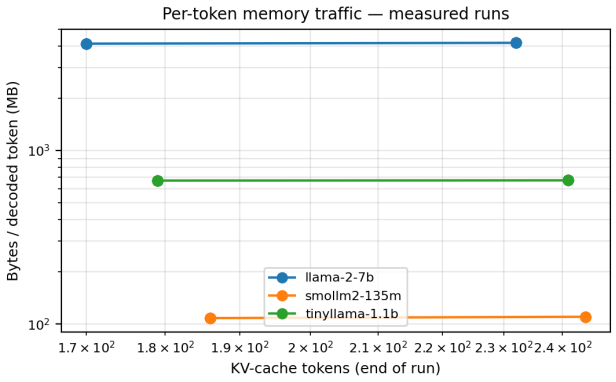


Fig. 6. Per-token memory traffic at decode (real measurements). Curves are dominated by the constant weight term until KV reads catch up; for Llama-2-7B at ctx = 4096, KV is still under 7 % of total per-token traffic.

### H. Energy per Token

Figure 7 stacks per-token energy contributions of weight reads, KV reads (at ctx = 2048), and MAC operations. Across all three models, the modeled memory contribution exceeds the compute contribution by more than two orders of magnitude. Under this model, reducing bytes moved is the highest-leverage optimization for energy efficiency.

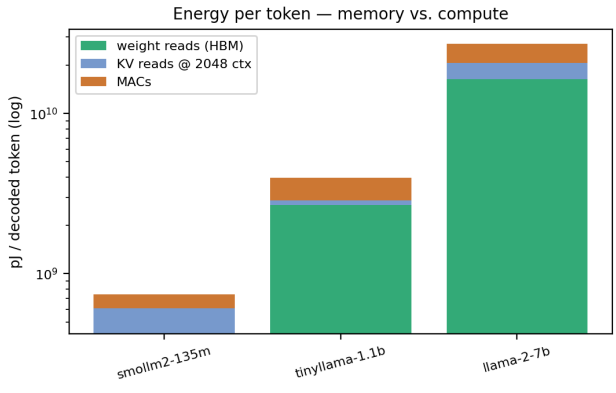


Fig. 7. Energy per decoded token, stacked by source. Memory dominates by >700× in every configuration tested.

### I. Simulator Validation

Figure 8 scatters modeled decode time against measured decode time on A100. Points cluster around $y = x$ with a systematic over-prediction by the simulator (the simulator assumes zero overlap between weight reads and KV reads; real implementations achieve partial overlap, hitting ∼45% of peak HBM as discussed in Sec. VI-D).

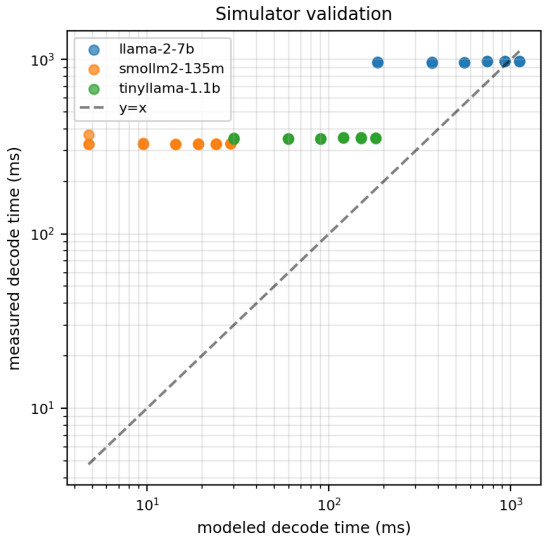


Fig. 8. Simulator validation: modeled vs. measured decode time across all real Llama-2-7B runs.

## VII. Analysis and Discussion

### A. Why the Adaptive Policy Matters Where It Does

The adaptive policy delivers a 46.8% tail-latency reduction at the HBM→DRAM crossing for Llama-2-7B, and zero benefit at the SRAM→HBM crossing for TinyLlama. This is exactly what the analytical model predicts: the adaptive win equals the migration cost spread over $K$ steps, divided by the per-step decode latency. When the migration is small (SRAM crossing, ∼4 MB at 10 TB/s = 0.4 $\mu$s), there is nothing

to amortize. When the migration is large (HBM crossing, $\sim$32 GB at 68 GB/s = 476 ms), the amortization is everything.

The system-design implication is that adaptive placement is most useful at wide bandwidth discontinuities. In our tier model, HBM$\rightarrow$DRAM is roughly a $50\times$ bandwidth drop, and DRAM$\rightarrow$CXL adds another $\sim$2.7$\times$ drop. A capacity-reactive baseline is acceptable when migration is small enough to be hidden in a decode step; at long contexts, the same assumption no longer holds.

### B. Why LLM Decode is Structurally Memory-Bound

Two decode properties make memory movement a persistent constraint. First, at batch 1 the weight stream is not amortized across many simultaneous tokens; each output token requires another pass over the weights. Second, the KV-cache grows with tokens, layers, and KV heads, and attention reads the relevant cache state during decode. Arithmetic grows with model size as well, so the ratio captured by AI remains near 2–3 FLOP/byte in our measurements, explaining the clustering in Fig. 2.

The cross-bandwidth experiment is consistent with this view. Llama-2-7B runs $4.21\times$ faster on A100 than on T4, while the published memory-bandwidth ratio is $4.86\times$ (87% match). This pair of GPUs has a similar ratio for tensor-core peak throughput, so bandwidth and compute ratios are not fully separable from this comparison alone; the low measured AI and the poorer scaling of smaller models provide the stronger evidence for the memory-bound diagnosis.

### C. Why the Memory-Bound Prediction Is Off by 2$\times$ (and Why the Gap Widens for Smaller Models)

Our memory-bound prediction is $\text{tps} = B_{\text{HBM, peak}}/b_{\text{tot}}$. For Llama-2-7B, measurements reach roughly 45% of this idealized prediction on both A100 and T4. We treat this as expected implementation loss: the analytical model assumes full use of peak bandwidth and no launch, scheduling, or pipeline overhead, while real kernels and runtime stacks do not sustain the theoretical peak for every decode step. The similar fractional gap across the two GPUs supports the interpretation that the model is identifying the right bottleneck even though it remains optimistic in absolute throughput.

For smaller models the gap widens: the memory-bound prediction overestimates by 4.4$\times$ for TinyLlama and by 30$\times$ for SmolLM2 on A100. We attribute this to runtime overhead per decode step that does not scale with model size: each step incurs a fixed cost in Python$\leftrightarrow$C++ FFI, kernel-launch latency, sampling, and bookkeeping. For Llama-2-7B at $\sim$4.5 GB per step, this fixed cost is amortised; for SmolLM2 at $\sim$135 MB per step, it is comparable to the bandwidth-bound estimate itself. This is a methodological caveat for memory-centric analysis at the sub-billion-parameter scale and suggests that bandwidth-focused architecture studies should be evaluated on models large enough for fixed runtime overhead to be amortized.

### D. The Cross-Bandwidth Result Strengthens with Model Size

Table V shows that the A100/T4 measured-throughput ratio matches the HBM-bandwidth ratio ($4.86\times$) most closely for the largest model: 87% for Llama-2-7B, 46% for TinyLlama, 33% for SmolLM2. This is the same phenomenon as the memory-bound prediction gap, viewed from the cross-bandwidth angle. The trend supports the paper's main claim: as model size increases, fixed runtime overhead is amortized over more bytes per decode step, and bandwidth-tracking becomes clearer. This is the regime where memory-centric placement policies are most likely to matter.

### E. Why KV-Quantization Looks Smaller Here Than Reported Elsewhere

KV-int4 reduces Llama-2-7B's bytes-per-token at ctx $= 4096$ by only 25.9% in our model. KVQuant [9] reports much larger speedups – but at much longer contexts (10M tokens). The reason is structural: at ctx $= 4096$ the KV-cache is small relative to the weight tensor (KV $\sim$2.0 GB vs. weights 4.08 GB), so even halving the KV term only saves a fraction of the total. As context grows past the crossover $T^* \approx 65$k tokens, KV dominates and KV quantization becomes the overwhelming optimization. Our model predicts this; we lack the GPU memory to measure it directly at ctx $\geq 65$k.

### F. Limitations

We acknowledge the following:

1) *Adaptive policy is simulator-only.* We do not modify `llama.cpp`'s C++ to implement the policy in production; the analyzer post-processes a real or synthetic decode trace. A production implementation would require a CUDA kernel that copies KV blocks across PCIe in the background of the decode kernel.
2) *No measured tier migrations.* The largest real run (4096 tokens) does not approach the HBM boundary on the GPUs we have. The headline adaptive-vs-static comparison is on a synthetic decode trace where the simulator drives the KV size to 65,792 tokens.
3) *No batching.* All runs use batch=1. Real serving systems batch decode steps across users, partially amortizing weight reads. The adaptive policy generalizes to batched decode with no structural change.
4) *Energy figures are derived from published per-byte and per-MAC numbers*, not measured power draw. The qualitative conclusion (memory $\gg$ compute) is robust to large changes in those constants.
5) *Per-step runtime overhead at small model sizes.* For SmolLM2 ($\sim$135 MB bytes/step), the fixed Python/C++/kernel-launch cost is a meaningful share of the decode budget, weakening the bandwidth-tracking signal in the cross-bandwidth experiment (Sec. VI-C). The memory-bound argument is cleanest at $\geq$1 B parameters; we report the small-model numbers transparently rather than excluding them.

## VIII. Conclusion and Future Work

This paper treats the timing of memory-tier migration as an explicit design variable for LLM inference. In the static baseline, capacity exhaustion is handled at the crossing step, so the modeled HBM→DRAM migration for Llama-2-7B contributes to a 1011.8 ms boundary step. The adaptive policy begins migration over the previous $K = 16$ decode steps; in the simulated trace, this reduces the worst-case step latency by 46.8% with no change in total decode time. The result suggests that temporal scheduling, even without reducing total bytes moved, can remove a major source of boundary-step tail latency.

We complemented the policy study with cross-bandwidth measurements across two NVIDIA accelerators ($4.86\times$ memory-bandwidth ratio), showing that Llama-2-7B throughput reaches 87% of the bandwidth-ratio prediction and that a compute-only throughput estimate overpredicts measured tok/s by two orders of magnitude on both hosts. We also placed KV quantization, sliding-window attention, and operator fusion under a single bytes-per-token model and found sliding-window $W = 512$ to be the largest single optimization at the tested context length.

**Future work:**

- *Production implementation.* Port the adaptive policy into `llama.cpp`'s CUDA backend or vLLM's PagedAttention manager, so the in-flight migration overlaps with weight reads on real silicon.
- *Online-learned $K$.* Replace the fixed lookahead with a controller that adjusts $K$ from observed decode-step latency variance.
- *Batched decode.* Extend the simulator to model multiple concurrent KV-caches and the cross-tenant scheduling problem that emerges.

The artefact is a single Colab notebook executable end-to-end on a Pro-tier runtime, published at https://github.com/mahima110298/Adaptive-KV-Cache-Placement-for-Tiered-Memory-LLM-Inference.git.

## REFERENCES


[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An Insightful Visual Performance Model for Multicore Architectures," *Commun. ACM*, vol. 52, no. 4, pp. 65–76, Apr. 2009.

[2] Y. Sheng et al., "FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU," in *Proc. ICML*, 2023.

[3] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[4] H. A. Maruf et al., "TPP: Transparent Page Placement for CXL-Enabled Tiered Memory," in *Proc. ASPLOS*, 2023.

[5] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," in *Proc. NeurIPS*, 2022.

[6] G. Gerganov et al., "llama.cpp: Efficient LLM Inference in C/C++," GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[7] JEDEC, "High Bandwidth Memory 3 (HBM3) DRAM Standard," JESD238, 2022.

[8] A. Gholami, Z. Yao, S. Kim, C. Hooper, M. W. Mahoney, and K. Keutzer, "AI and Memory Wall," *IEEE Micro*, 2024.

[9] C. Hooper et al., "KVQuant: Towards 10 Million Context Length LLM Inference with KV Cache Quantization," in *Proc. NeurIPS*, 2024.

[10] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in *Proc. ICLR*, 2024.

[11] Compute Express Link Consortium, "Compute Express Link (CXL) Specification, Revision 3.0," 2022. [Online]. Available: https://www.computeexpresslink.org/

[12] M. Horowitz, "Computing's Energy Problem (and What We Can Do About It)," in *Proc. IEEE ISSCC*, 2014, pp. 10–14.

[13] A. Betlen, "llama-cpp-python: Python Bindings for llama.cpp," GitHub repository, 2023–2024. [Online]. Available: https://github.com/abetlen/llama-cpp-python

[14] H. Touvron et al., "Llama 2: Open Foundation and Fine-Tuned Chat Models," *arXiv:2307.09288*, 2023.

[15] K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated Memory for Expansion and Sharing in Blade Servers," in *Proc. ACM ISCA*, 2009, pp. 267–278.

[16] NVIDIA, "NVIDIA A100 Tensor Core GPU Architecture," white paper, 2020. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf

[17] NVIDIA, "NVIDIA T4 70W Low Profile PCIe GPU Accelerator," product brief, 2020. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-product-brief.pdf

[18] NVIDIA, "NVIDIA H100 Tensor Core GPU," datasheet, 2023. [Online]. Available: https://www.nvidia.com/en-us/data-center/h100/

[19] JEDEC, "DDR5 SDRAM Standard," JESD79-5, 2020.

[20] Hugging Face, "Chat Templates," *Hugging Face LLM Course*, 2024. [Online]. Available: https://huggingface.co/learn/llm-course/en/chapter11/2

[21] HuggingFaceTB, "SmolLM2-135M-Instruct," model card and tokenizer configuration, Hugging Face, 2024. [Online]. Available: https://huggingface.co/HuggingFaceTB/SmolLM2-135M-Instruct

[22] TinyLlama Team, "TinyLlama-1.1B-Chat-v1.0," model card, Hugging Face, 2023. [Online]. Available: https://huggingface.co/TinyLlama/TinyLlama-1.1B-Chat-v1.0

[23] Meta Llama, "Llama 2 Model Card," GitHub repository, 2023. [Online]. Available: https://github.com/meta-llama/llama-models/blob/main/models/llama2/MODEL_CARD.md

[24] ggml-org, “llama.cpp quantize tool,” GitHub repository, 2024–2026. [Online]. Available: https://github.com/ggml-org/llama.cpp/blob/master/tools/quantize/README.md